Banking Application — Architecture & Design Document

Version: 2.0
Date: April 15, 2026
Author: Architecture Team
Status: Comprehensive

Table of Contents

1. System Overview & High-Level Architecture
2. Core Modules — Design & Implementation
3. Additional Modules (Admin, Reports, Scheduler, Disputes)
4. Database Schema & Data Model
5. Event-Driven Architecture with Apache Kafka
6. Kafka Dead Letter Queue & Error Handling
7. Caching Strategy
8. Multi-Level Filter Implementation
9. Concurrency Control & Double-Spend Prevention
10. Service Discovery & Inter-Service Communication
11. Resilience Patterns
12. Security Architecture
13. API Design Contract
14. Frontend Architecture
15. Observability & Monitoring
16. CI/CD Pipeline & Testing Strategy
17. Deployment & Infrastructure
18. Disaster Recovery & Business Continuity
19. Regulatory Compliance
20. End-to-End Request Lifecycle

---

1. System Overview & High-Level Architecture

The application follows a **microservices architecture** with an **event-driven backbone** powered by Apache Kafka. The frontend is a React SPA communicating through a BFF (Backend-for-Frontend) layer and API Gateway, which routes requests to domain-specific backend services. Redis provides multi-layer caching, and PostgreSQL serves as the primary data store.

1.1 High-Level Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                                   CLIENTS                                    │
│                                                                              │
│          ┌──────────────┐     ┌──────────────┐     ┌──────────────┐          │
│          │  Web (React) │     │  Mobile App  │     │  Third-Party │          │
│          └──────────────┘     └──────────────┘     └──────────────┘          │
└──────────────────┬──────────────────────┬──────────────────────┬──────────────┘
                   │                      │                      │
                   ▼                      ▼                      ▼

┌──────────────────────────────────────────────────────────────────────────────┐
│                              EDGE LAYER                                      │
│  ┌──────────────┐  ┌──────────────────────────────────────────────────────┐  │
│  │    WAF       │  │                CDN (CloudFront)                      │  │
│  │ (AWS WAF)    │  │  Static assets, TLS termination                      │  │
│  └──────┬───────┘  └──────────────────────────────────────────────────────┘  │
│         ▼                                                                    │
│  ┌──────────────────────────────────────────────────────────────────────┐    │
│  │              API GATEWAY (Kong / Spring Cloud Gateway)               │    │
│  │    ┌──────────┬──────────┬──────────┬──────────┬──────────┐         │    │
│  │    │Rate Limiting│JWT Validate│Load Balancing│ CORS/CSRF │ Request Log │    │
│  │    └──────────┴──────────┴──────────┴──────────┴──────────┘         │    │
│  └──────────────────────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────────────────────┘
        │                      │
        ▼                      ▼

┌──────────────┐     ┌──────────────┐
│  BFF - Web   │     │ BFF - Mobile │    (Backend-for-Frontend - tailored payloads)
└──────────────┘     └──────────────┘
        │                      │
        ▼                      ▼

┌──────────────────────────────────────────────────────────────────────────────┐
│                    SERVICE MESH (Istio - mTLS between all services)          │
│                                                                              │
│  ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐ │
│  │  Auth    │ │ Account  │ │Transaction│ │ Payment  │ │   Loan   │ │   Admin  │ │
│  │ Service  │ │ Service  │ │  Service  │ │ Service  │ │ Service  │ │ Service  │ │
│  │          │ │          │ │           │ │          │ │          │ │          │ │
│  │- Login   │ │- Create  │ │- Transfer │ │- UPI     │ │- Apply   │ │- User Mgmt│ │
│  │- OAuth   │ │- Balance │ │- History  │ │- NEFT    │ │- EMI Calc│ │- Dispute │ │
│  │- MFA     │ │- KYC     │ │- Statement│ │- RTGS    │ │- Disburse│ │- Override│ │
│  │- RBAC    │ │- Freeze  │ │- Filters  │ │- Bill Pay│ │- Repay   │ │- Reports │ │
│  └──────────┘ └──────────┘ └──────────┘ └──────────┘ └──────────┘ └──────────┘ │
│       │           │            │            │           │            │        │
│  ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐  │
│  │Notification│ │  Fraud   │ │ Scheduler│ │Reporting │ │  Config  │  │
│  │ Service  │ │ Detection│ │ Service  │ │ Service  │ │ Service  │  │
│  │          │ │ Service  │ │          │ │          │ │(Spring Cloud)│  │
│  │- SMS     │ │- Rule Eng.│ │- Recurring│ │- Stmts   │ │- Feature flags│  │
│  │- Email   │ │- ML Scoring│ │- EMI auto-dbt│ │- Analytics│ │- Rate tables│  │
│  │- Push    │ │- Alerts  │ │- EOD batches│ │- PDF gen │ │- Env configs│  │
│  └──────────┘ └──────────┘ └──────────┘ └──────────┘ └──────────┘  │
└──────────────────────────────────────────────────────────────────────────────┘
            │                      │
            │                      │
```

```
                ▼                        ▼
┌──────────────────────────────────────────────────────────────────────┐
│                        EVENT BUS (Apache Kafka)                      │
│                                                                      │
│   ┌─────────────────┐  ┌─────────────────┐  ┌─────────────────┐  ┌─────────────────┐
│   │ txn-events      │  │ account-events  │  │ notification-   │  │ audit-events    │
│   │ (12 partitions)│  │ (6 partitions)  │  │ events (6 pt)   │  │ (6 partitions)  │
│   └─────────────────┘  └─────────────────┘  └─────────────────┘  └─────────────────┘
│   ┌─────────────────┐  ┌─────────────────┐  ┌─────────────────┐
│   │ fraud-events    │  │ scheduler-      │  │ *.DLQ           │
│   │ (6 partitions)  │  │ events (3 pt)   │  │ (Dead Letters)  │
│   └─────────────────┘  └─────────────────┘  └─────────────────┘
└──────────────────────────────────────────────────────────────────────┘
                |
                ▼
┌──────────────────────────────────────────────────────────────────────┐
│                              DATA LAYER                              │
│                                                                      │
│   ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐
│   │  PostgreSQL  │  │    Redis     │  │   MongoDB    │  │ Elasticsearch│
│   │  (Primary DB) │  │   (Cache)    │  │ (Audit Logs) │  │  (Search)    │
│   │              │  │              │  │              │  │              │
│   │ • Accounts   │  │ • Sessions   │  │ • Audit Trail│  │ • Txn Search │
│   │ • Txns       │  │ • Balances   │  │ • Login Logs │  │ • Full-text  │
│   │ • Loans      │  │ • OTP Cache  │  │ • API Logs   │  │ • Analytics  │
│   │ • Users      │  │ • Rate Limits│  │ • Disputes   │  │              │
│   │ • Sagas      │  │ • Locks      │  │              │  │              │
│   └──────────────┘  └──────────────┘  └──────────────┘  └──────────────┘
│
│   ┌──────────────────────┐  ┌──────────────────────┐
│   │   HashiCorp Vault    │  │   AWS S3             │
│   │   (Secrets Mgmt)     │  │   (Statement PDFs,   │
│   │   • DB credentials   │  │    backups, archives)  |
│   │   • API keys         │  │                      │
│   │   • Encryption keys  │  │                      │
│   └──────────────────────┘  └──────────────────────┘
└──────────────────────────────────────────────────────────────────────┘
```

2. Core Modules — Design & Implementation

2.1 Module Overview Table

Module	Tech Stack	Database	Kafka Topics	Cache Strategy
Auth Service	Spring Boot + Spring Security	PostgreSQL	`audit-events`	Redis (sessions, OTP)
Account Service	Spring Boot + JPA	PostgreSQL	`account-events`	Redis (balances)
Transaction Svc	Spring Boot + Saga Orchestrator	PostgreSQL	`txn-events`	Redis (recent txns)
Payment Service	Spring Boot + Integration Layer	PostgreSQL	`txn-events`, `notification`	Redis (idempotency)
Loan Service	Spring Boot + Rule Engine	PostgreSQL	`account-events`	Local (rate tables)
Notification	Spring Boot (Kafka Consumer)	MongoDB	`notification-events`	Redis (templates)
Fraud Detection	Python + ML Pipeline	MongoDB	`fraud-events`, `txn-events`	Redis (risk scores)
Admin Service	Spring Boot + Thymeleaf	PostgreSQL	`audit-events`	Redis (user cache)
Reporting Svc	Spring Boot + Jasper/PDFBox	PG Read Rep.	`txn-events` (materialized)	Redis (report cache)
Scheduler Svc	Spring Boot + Quartz	PostgreSQL	`scheduler-events`	None
Config Service	Spring Cloud Config Server	Git repo	—	Local (refresh bus)

2.2 Auth Service

Handles authentication (OAuth 2.0 + JWT), multi-factor authentication, and role-based access control (RBAC).

```
                        Auth Flow
                        ─────────

Client                  Auth Service                  Redis              Vault
  │                         │                           │                  │
  │── POST /login ──────────►│                           │                  │
  │                         │── Fetch signing key ──────────────────────────►│
  │                         │── Validate credentials    │                  │
  │                         │── Generate OTP ──────────►│  SET otp:{uid} EX 300
  │◄── 200 (OTP sent) ──────│                           │                  │
  │                         │                           │                  │
  │── POST /verify-otp ─────►│                           │                  │
  │                         │── GET otp:{uid} ──────────►│                  │
  │                         │◄── OTP value ─────────────│                  │
  │                         │── Validate & Issue JWT    │                  │
  │                         │── SET session:{tok} ──────►│  EX 3600         │
  │◄── 200 (JWT + Refresh)──│                           │                  │
  │                         │── PUBLISH audit-events ──► Kafka             │
```

Implementation details:

- Passwords stored with **BCrypt** (cost factor 12).
- JWT tokens contain `userId`, `roles[]`, and `accountIds[]` — signed with **RS256**. Private key stored in Vault, rotated every 90 days.
- Refresh tokens stored in Redis with a 7-day TTL; access tokens expire in 15 minutes.
- MFA via TOTP (Google Authenticator) or SMS OTP cached in Redis for 5 minutes.
- **RBAC roles:** `CUSTOMER`, `TELLER`, `MANAGER`, `ADMIN`, `AUDITOR` — each with fine-grained permissions.
- Failed login attempts tracked in Redis. Account locked after 5 failures within 15 minutes.

2.3 Transaction Service (Saga Pattern)

Fund transfers use the **Saga Orchestrator Pattern** to maintain consistency across services without distributed transactions.

```
                    Transfer Saga — Orchestrated
                    ────────────────────────────


┌──────────────┐     ┌──────────────┐     ┌──────────────┐     ┌──────────────┐
│  Transaction  │     │   Account    │     │   Account    │     │ Notification │
│ Orchestrator  │     │  (Debit)     │     │  (Credit)    │     │   Service    │
└──────────────┘     └──────────────┘     └──────────────┘     └──────────────┘
       │                    │                    │                    │
  0.   │── Acquire distributed lock (Redis) ──►│                    │
       │── Idempotency check (Redis SET NX) ──►│                    │
       │                    │                    │                    │
  1.   │── Debit ──────────►│                    │                    │
       │                    │── SELECT FOR UPDATE │                    │
       │                    │── Check balance ≥ amt                    │
       │                    │── Deduct Balance   │                    │
       │◄── Debited ────────│                    │                    │
       │                    │                    │                    │
  2.   │── Credit ──────────┼──────────────────►│                    │
       │                    │                    │── SELECT FOR UPDATE │
       │                    │                    │── Add Balance      │
       │◄── Credited ───────┼────────────────────│                    │
       │                    │                    │                    │
  3.   │── Persist saga state (COMPLETED) to PG  │                    │
       │── Release distributed lock ──────────►│                    │
       │── PUBLISH txn.completed ──────────────────────────────────► Kafka
       │                    │                    │                    │
  4.   │                    │                    │        ◄── CONSUME │
       │                    │                    │         Send SMS   │
       │                    │                    │         Send Email │

  ── On failure at Step 2: ──
       │── Compensate: Reverse Debit ──►│  (with same idempotency key)
       │── Persist saga state (COMPENSATED)
       │── PUBLISH txn.failed ──────────────────────────────────► Kafka
```

Implementation details:

- Each saga step is idempotent — retries are safe via an **idempotency key** stored in Redis with 24h TTL.
- Saga state machine persisted in PostgreSQL (`saga_state` table) for crash recovery.
- Kafka guarantees at-least-once delivery; consumer deduplication via the idempotency key.
- **Distributed lock** (Redis `SET accountId:lock NX EX 10`) prevents concurrent modifications to the same account.

2.4 Payment Service

Integrates with external payment networks:

Payment Type	External Integration	Protocol	SLA
UPI	NPCI (National Payments Corporation)	REST API + Callback	Real-time
NEFT	RBI NEFT system	ISO 20022 XML	Batch (hourly)
RTGS	RBI RTGS system	ISO 20022 XML	Real-time (≥ ₹2L)
IMPS	NPCI IMPS	REST API	Real-time
Bill Pay	BBPS (Bharat BillPay)	REST API	Near real-time
International	SWIFT network	MT103 / ISO 20022	1-3 business days

2.5 Loan Service

- **Rule Engine** (Drools) evaluates eligibility based on credit score, income, existing EMIs.
- **Credit Bureau Integration:** CIBIL/Experian API call (with 10s timeout + circuit breaker).
- **eKYC Integration:** Aadhaar-based OTP verification via UIDAI API.
- EMI calculation uses reducing-balance method; schedule persisted to `loan_schedule` table.

---

3. Additional Modules

3.1 Admin / Back-Office Service

Internal-facing portal for operations and compliance teams.

```
┌──────────────────────────────────────────────────────────┐
│                      ADMIN PORTAL                        │
│                                                          │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐   │
│  │ User Mgmt    │  │  Dispute     │  │  Operations  │   │
│  │              │  │  Handling    │  │              │   │
│  │ • Search user│  │ • View claim │  │ • Freeze acct│   │
│  │ • Lock/Unlock│  │ • Assign     │  │ • Manual txn │   │
│  │ • Role assign│  │ • Investigate│  │ • Override   │   │
│  │ • KYC review │  │ • Resolve    │  │ • Bulk ops   │   │
│  │ • Audit trail│  │ • Refund saga│  │ • EOD recon  │   │
│  └──────────────┘  └──────────────┘  └──────────────┘   │
│                                                          │
│  ┌──────────────┐  ┌──────────────┐                     │
│  │  Compliance  │  │   Config     │                     │
│  │              │  │   Manager    │                     │
│  │ • CTR reports│  │ • Rate tables│                     │
│  │ • SAR filing │  │ • Fee config │                     │
│  │ • Reg reports│  │ • Limits     │                     │
│  │ • Data export│  │ • Feature flg│                     │
│  └──────────────┘  └──────────────┘                     │
└──────────────────────────────────────────────────────────┘
```

Access control: Requires `ADMIN` or `MANAGER` role. Every action emits an `audit-events` Kafka message. All admin actions require a **maker-checker** workflow (one person initiates, another approves).

3.2 Dispute & Chargeback Workflow

```
Customer                Admin Portal            Transaction Svc         Notification
   |                        |                        |                        |
   |── Raise dispute ──────►|                        |                        |
   |                        |── Create case (OPEN)   |                        |
   |                        |── Assign to agent      |                        |
   |                        |                        |                        |
   |                        |── Investigate          |                        |
   |                        |── GET /txn/{id}/details►|                        |
   |                        |◄── Full txn trail ─────|                        |
   |                        |                        |                        |
   |                 ┌──────┴──────┐                 |                        |
   |                 │  Decision   │                 |                        |
   |                 ├─────────────┤                 |                        |
   |                 │  APPROVE    │──── Refund Saga ►|── Reverse txn          |
   |                 │  REJECT     │                 |                        |
   |                 └─────────────┘                 |                        |
   |                        |── Update case (CLOSED) |                        |
   |                        |── PUBLISH notification ──────────────────────►|
   |◄── Email: Resolution ──────────────────────────────────── Send email ───|
```

States: `OPEN → ASSIGNED → UNDER_REVIEW → APPROVED/REJECTED → CLOSED`

3.3 Reporting & Analytics Service

- Reads from **PostgreSQL read replicas** to avoid impacting transactional workload.
- Uses **CQRS pattern**: writes go to primary DB via event handlers; reads served from denormalized views.
- **Statement PDF generation** via Apache PDFBox, stored in S3, download link sent via notification.
- **Materialized views** refreshed every 15 minutes for dashboard aggregates (daily totals, category breakdown).

```sql
-- Materialized view for daily transaction summary (refreshed by Scheduler Service)
CREATE MATERIALIZED VIEW mv_daily_txn_summary AS
SELECT
    account_id,
    DATE(created_at) AS txn_date,
    COUNT(*)         AS txn_count,
    SUM(CASE WHEN type = 'CREDIT' THEN amount ELSE 0 END) AS total_credit,
    SUM(CASE WHEN type = 'DEBIT'  THEN amount ELSE 0 END) AS total_debit
FROM transactions
WHERE created_at ≥ CURRENT_DATE - INTERVAL '90 days'
GROUP BY account_id, DATE(created_at);

CREATE UNIQUE INDEX idx_mv_daily ON mv_daily_txn_summary (account_id, txn_date);
```

3.4 Scheduler Service

Built on **Quartz Scheduler** with a persistent job store (PostgreSQL), so jobs survive pod restarts.

Job	Schedule	Description
EMI Auto-Debit	Daily 6:00 AM	Debit EMI from linked accounts
Standing Instructions	Per-instruction	Recurring transfers (rent, SIP, etc.)
EOD Reconciliation	Daily 11:30 PM	Match internal ledger with CBS
Materialized View Refresh	Every 15 min	Refresh `mv_daily_txn_summary`
Statement Generation	Monthly, 1st	Generate PDF statements for all accounts
Dormant Account Check	Weekly	Flag accounts inactive > 2 years
Kafka Lag Monitor	Every 5 min	Alert if consumer lag exceeds threshold

3.5 Config Service (Spring Cloud Config)

Centralized configuration management:

- Config stored in a **Git repository** (versioned, auditable).
- Services fetch config on startup and listen on `/actuator/bus-refresh` for hot-reload.
- **Feature flags** managed via config: e.g., `feature.upi-autopay.enabled=true`.
- Environment-specific overlays: `application-dev.yml`, `application-staging.yml`, `application-prod.yml`.
- Sensitive values (DB passwords, API keys) are **NOT** in config — they come from Vault.

3.6 BFF (Backend-for-Frontend) Layer

Two BFF services tailor API responses for different clients:

BFF	Optimizations
BFF-Web	Full payloads, pagination, WebSocket connection for live updates
BFF-Mobile	Compressed payloads, aggressive field filtering, push notification tokens, offline-sync metadata

Both BFFs aggregate calls to downstream services (e.g., Dashboard = Account Balance + Recent 5 Txns + Loan EMI due — single API call from client, 3 parallel gRPC calls internally).

---

4. Database Schema & Data Model

4.1 Service-to-Database Ownership

Each microservice owns its database (database-per-service pattern). No service reads another service's tables directly.

```
┌──────────────────┐      ┌──────────────────┐      ┌──────────────────┐
│  Auth Service    │      │ Account Service  │      │Transaction Svc   │
│                  │      │                  │      │                  │
│  ┌────────────┐  │      │  ┌────────────┐  │      │  ┌────────────┐  │
│  │ auth_db    │  │      │  │account_db  │  │      │  │  txn_db    │  │
│  │            │  │      │  │            │  │      │  │            │  │
│  │• users     │  │      │  │• accounts  │  │      │  │• transact  │  │
│  │• roles     │  │      │  │• kyc_docs  │  │      │  │• saga_st   │  │
│  │• perms     │  │      │  │• nominees  │  │      │  │• idempot   │  │
│  │• mfa_conf  │  │      │  │• mandates  │  │      │  │            │  │
│  └────────────┘  │      │  └────────────┘  │      │  └────────────┘  │
└──────────────────┘      └──────────────────┘      └──────────────────┘

┌──────────────────┐      ┌──────────────────┐      ┌──────────────────┐
│ Payment Service  │      │  Loan Service    │      │ Scheduler Svc    │
│                  │      │                  │      │                  │
│  ┌────────────┐  │      │  ┌────────────┐  │      │  ┌────────────┐  │
│  │payment_db  │  │      │  │ loan_db    │  │      │  │scheduler   │  │
│  │            │  │      │  │            │  │      │  │  _db       │  │
│  │• payments  │  │      │  │• loans     │  │      │  │• qrtz_*    │  │
│  │• benefic   │  │      │  │• emi_sched │  │      │  │• job_hist  │  │
│  │• ext_refs  │  │      │  │• disburse  │  │      │  │            │  │
│  └────────────┘  │      │  └────────────┘  │      │  └────────────┘  │
└──────────────────┘      └──────────────────┘      └──────────────────┘
```

4.2 Core Entity-Relationship Diagram

```
┌──────────────┐        ┌──────────────────┐        ┌──────────────────┐
│    users     │        │    accounts      │        │  transactions    │
├──────────────┤        ├──────────────────┤        ├──────────────────┤
│ id (PK)      │──┐     │ id (PK)          │──┐     │ id (PK, UUID)    │
│ email (UQ)   │  │     │ user_id (FK)     │  │     │ account_id (FK)  │
│ phone (UQ)   │  │     │ account_number(UQ)│  │     │ contra_acct_id   │
│ password_hash│  └──1:N│ type (SAVINGS/   │  └──1:N│ amount (NUMERIC) │
│ status       │        │   CURRENT/LOAN)  │        │ currency         │
│ created_at   │        │ balance (NUMERIC) │        │ type (CR/DR)     │
│ updated_at   │        │ currency         │        │ category         │
│ version (*)  │        │ status (ACTIVE/  │        │ status (SUCCESS/ │
└──────────────┘        │   FROZEN/DORMANT) │        │  PENDING/FAILED) │
       │                │ opened_at        │        │ description      │
       │                │ version (*)      │        │ reference_id     │
┌──────┴───────┐        └──────────────────┘        │ idempotency_key  │
│  user_roles  │               │                    │ created_at       │
├──────────────┤               │                    │ updated_at       │
│ user_id (FK) │        ┌──────┴───────────┐        └──────────────────┘
│ role (ENUM)  │        │   nominees       │
│ granted_at   │        ├──────────────────┤        ┌──────────────────┐
│ granted_by   │        │ id (PK)          │        │   saga_state     │
└──────────────┘        │ account_id (FK)  │        ├──────────────────┤
                        │ name             │        │ id (PK, UUID)    │
┌──────────────┐        │ relationship     │        │ saga_type        │
│  mfa_config  │        │ share_pct        │        │ status (STARTED/ │
├──────────────┤        └──────────────────┘        │  DEBIT_DONE/     │
│ user_id (FK) │                                    │  COMPLETED/      │
│ type (TOTP/  │        ┌──────────────────┐        │  COMPENSATING/   │
│   SMS)       │        │     loans        │        │  COMPENSATED)    │
│ secret_ref   │        ├──────────────────┤        │ payload (JSONB)  │
│ enabled      │        │ id (PK)          │        │ current_step     │
└──────────────┘        │ user_id (FK)     │        │ created_at       │
                        │ account_id (FK)  │        │ updated_at       │
                        │ principal        │        └──────────────────┘
                        │ interest_rate    │
                        │ tenure_months    │        ┌──────────────────┐
                        │ emi_amount       │        │  idempotency_key │
                        │ status (APPLIED/ │        ├──────────────────┤
                        │  APPROVED/ACTIVE/│        │ key (PK)         │
                        │  CLOSED/DEFAULTED│        │ response (JSONB) │
                        │ disbursed_at     │        │ created_at       │
                        │ version (*)      │        │ expires_at       │
                        └──────────────────┘        └──────────────────┘
                               │
                        ┌──────┴───────────┐        ┌──────────────────┐
                        │  loan_schedule   │        │    disputes      │
                        ├──────────────────┤        ├──────────────────┤
                        │ id (PK)          │        │ id (PK)          │
                        │ loan_id (FK)     │        │ txn_id (FK)      │
                        │ installment_no   │        │ raised_by (FK)   │
                        │ due_date         │        │ assigned_to (FK) │
                        │ emi_amount       │        │ status (OPEN/    │
```

```
                          | principal_part    |        |  ASSIGNED/REVIEW/|
                          | interest_part     |        |  APPROVED/REJECT/|
                          | status (DUE/PAID/|        |  CLOSED)         |
                          |  OVERDUE)         |        | reason           |
                          | paid_at           |        | resolution       |
                          └───────────────────┘        | created_at       |
                                                       | resolved_at      |
  (*) version column = optimistic locking              └──────────────────┘
```

4.3 Transaction Table Partitioning

The `transactions` table will grow to billions of rows. We use **range partitioning by month**:

```sql
CREATE TABLE transactions (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    account_id      UUID NOT NULL REFERENCES accounts(id),
    amount          NUMERIC(18,2) NOT NULL,
    type            VARCHAR(10) NOT NULL,       -- CREDIT / DEBIT
    category        VARCHAR(20) NOT NULL,       -- UPI / NEFT / RTGS / ATM / EMI / BILL_
    status          VARCHAR(10) NOT NULL,       -- SUCCESS / PENDING / FAILED
    description     TEXT,
    reference_id    VARCHAR(64),
    idempotency_key VARCHAR(64) UNIQUE,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
) PARTITION BY RANGE (created_at);

-- Auto-create monthly partitions
CREATE TABLE transactions_2026_01 PARTITION OF transactions
    FOR VALUES FROM ('2026-01-01') TO ('2026-02-01');
CREATE TABLE transactions_2026_02 PARTITION OF transactions
    FOR VALUES FROM ('2026-02-01') TO ('2026-03-01');
--  ... (automated via pg_partman extension)

-- Indexes (created per partition automatically)
CREATE INDEX idx_txn_account_date ON transactions (account_id, created_at DESC);
CREATE INDEX idx_txn_status       ON transactions (status) WHERE status ≠ 'SUCCESS';
CREATE INDEX idx_txn_category     ON transactions (category);
CREATE INDEX idx_txn_amount       ON transactions (amount);
CREATE INDEX idx_txn_description  ON transactions USING gin(to_tsvector('english', descr
```

4.4 Database Migration Strategy

- **Flyway** manages all schema migrations.
- Migration files follow: `V{version}__{description}.sql` (e.g., `V1__create_accounts_table.sql`).
- Migrations run automatically on service startup (before accepting traffic).
- **Backward-compatible only:** new columns are always nullable or have defaults. Old columns are never renamed or removed in the same release as the code change — always a two-phase

migration.

---

5. Event-Driven Architecture with Apache Kafka

5.1 Why Kafka in Banking?

Concern	How Kafka Solves It
Decoupling	Services communicate via topics, not direct HTTP calls
Audit Trail	Kafka log retention (30 days) acts as an immutable event ledger
Scalability	Partition-based parallelism handles millions of txns/day
Fault Tolerance	Replication factor 3 ensures no data loss on broker failure
Ordering	Partition key = `accountId` guarantees per-account event ordering
Replay	Consumers can reset offsets to reprocess historical events

5.2 Kafka Topology

```
 PRODUCERS                          KAFKA CLUSTER                        CONSUMERS
___________                       _______________                      ___________

Transaction  ──►  ┌──────────────────────────────────┐
  Service         │  txn-events (12 partitions)      │──►  Fraud Detection Service
                  │  Key: accountId                  │──►  Notification Service
                  │  Retention: 30 days              │──►  Analytics Pipeline
                  │  acks: all, min.isr: 2           │──►  Reporting Service
                  └──────────────────────────────────┘

Account      ──►  ┌──────────────────────────────────┐
  Service         │  account-events (6 partitions)   │──►  Loan Service
                  │  Key: accountId                  │──►  Notification Service
                  │  Retention: 30 days              │──►  Cache Invalidation Consumer
                  └──────────────────────────────────┘

Auth         ──►  ┌──────────────────────────────────┐
  Service         │  audit-events (6 partitions)     │──►  Audit Log Writer (→ MongoDB)
                  │  Key: userId                     │──►  Fraud Detection Service
                  │  Retention: 90 days (compliance) │
                  └──────────────────────────────────┘

All          ──►  ┌──────────────────────────────────┐
  Services        │  notification-events (6 partitions) │──►  Notification Service
                  │  Key: userId                     │     (SMS, Email, Push)
                  └──────────────────────────────────┘

Scheduler    ──►  ┌──────────────────────────────────┐
  Service         │  scheduler-events (3 partitions) │──►  Transaction Service
                  │  Key: jobId                      │──►  Reporting Service
                  └──────────────────────────────────┘

* Failed *   ──►  ┌──────────────────────────────────┐
                  │  *.DLQ (per-topic dead letter queues) │──►  DLQ Monitor / Alerting
                  │  Retention: 90 days              │──►  Manual Reprocessing Tool
                  └──────────────────────────────────┘
```

5.3 Event Schema (Avro + Schema Registry)

```json
{
  "eventId": "uuid-v4",
  "eventType": "TRANSACTION_COMPLETED",
  "timestamp": "2026-04-15T10:30:00Z",
  "aggregateId": "account-123",
  "payload": {
    "transactionId": "txn-456",
    "fromAccount": "ACC-001",
    "toAccount": "ACC-002",
    "amount": 5000.00,
    "currency": "INR"
  },
  "metadata": {
    "correlationId": "req-789",
    "causationId": "cmd-101",
    "sourceService": "transaction-service",
    "version": 1,
    "traceId": "jaeger-trace-abc",
    "userId": "user-007"
  }
}
```

All events are validated against **Confluent Schema Registry** (Avro format) to enforce backward compatibility. Schema evolution rules: fields can be added (with defaults), never removed or renamed. Consumer groups are named by service (e.g., `fraud-detection-cg`) so each service processes events independently.

---

6. Kafka Dead Letter Queue & Error Handling

6.1 Retry & DLQ Flow

```
                        Consumer Processing Flow
                        ────────────────────────

Kafka Topic              Consumer                  Retry Topic            DLQ Topic
───────────              ────────                  ───────────            ─────────
     │                       │                         │                      │
     │── Message ────────────►│                         │                      │
     │                       │── Process               │                      │
     │                       │                         │                      │
     │                  ┌────────┐                     │                      │
     │                  │ Success? │                    │                      │
     │                  └────────┘                     │                      │
     │                  YES   │  NO                    │                      │
     │                    │   │                        │                      │
     │             Commit│    │── Retry #1 (1s delay)  │                      │
     │             offset│    │── Retry #2 (5s delay)  │                      │
     │                    │   │── Retry #3 (30s delay) │                      │
     │                    │   │                        │                      │
     │                    │   │── Still failing? ──────►│                      │
     │                    │   │                        │── Retry #4 (5m)      │
     │                    │   │                        │── Retry #5 (30m)     │
     │                    │   │                        │── Retry #6 (2h)      │
     │                    │   │                        │                      │
     │                    │   │                        │── Still failing? ───►│
     │                    │   │                        │                      │── Store
     │                    │   │                        │                      │── Alert
     │                    │   │                        │                      │── Manual
     │                    │   │                        │                      │   Review
```

6.2 Retry Policy

Stage	Max Retries	Backoff Strategy	Delay Range
In-memory	3	Exponential	1s → 5s → 30s
Retry topic	3	Exponential	5m → 30m → 2h
DLQ	0	Manual reprocessing only	—

6.3 DLQ Monitoring

```java
@KafkaListener(topics = "txn-events.DLQ", groupId = "dlq-monitor-cg")
public void onDeadLetter(ConsumerRecord<String, GenericRecord> record) {
    String reason = new String(record.headers().lastHeader("error-reason").value());

    dlqMetrics.increment(record.topic(), reason);

    alertService.sendPagerDuty(Alert.builder()
        .severity(Severity.HIGH)
        .title("DLQ message: " + record.topic())
        .details(Map.of(
            "originalTopic", extractOriginalTopic(record),
            "key", record.key(),
            "reason", reason,
            "retryCount", extractRetryCount(record)
        ))
        .build());

    dlqRepository.save(DlqRecord.from(record));
}
```

6.4 Poison Pill Handling

Messages that cause deserialization errors (corrupt/incompatible schema) bypass retry entirely:

- Logged to a dedicated `poison-pill` MongoDB collection with the raw bytes.
- `ErrorHandlingDeserializer` wraps the value deserializer, catches exceptions, and routes to DLQ immediately.
- Alert fires with `severity=CRITICAL` — schema mismatch indicates a deployment issue.

---

7. Caching Strategy — Where, What, and How

7.1 Cache Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              CACHE LAYERS                                 │
│                                                                           │
│   LAYER 1: Browser / CDN                                                  │
│   ┌──────────────────────────────────────────────────────────────────┐   │
│   │  • Static assets (JS, CSS, images) — CDN with 1-year TTL         │   │
│   │  • API responses — Cache-Control: no-store (sensitive data)      │   │
│   └──────────────────────────────────────────────────────────────────┘   │
│                                                                           │
│   LAYER 2: API Gateway Cache                                              │
│   ┌──────────────────────────────────────────────────────────────────┐   │
│   │  • Rate limit counters — Redis (INCR + EXPIRE)                   │   │
│   │  • Idempotency keys — Redis SET NX EX 86400                      │   │
│   │  • Public endpoints (exchange rates, branch info) — 5 min        │   │
│   └──────────────────────────────────────────────────────────────────┘   │
│                                                                           │
│   LAYER 3: Application-Level Cache (Per Service)                          │
│   ┌──────────────────────────────────────────────────────────────────┐   │
│   │                                                                  │   │
│   │   ┌──────────────────────┐     ┌──────────────────────────────┐  │   │
│   │   │ LOCAL (Caffeine)     │     │ DISTRIBUTED (Redis)          │  │   │
│   │   │                      │     │                              │  │   │
│   │   │ • Interest rates     │     │ • Account balances   60s     │  │   │
│   │   │ • Loan rate tables   │     │ • User sessions      15m     │  │   │
│   │   │ • Branch metadata    │     │ • OTP codes          5m      │  │   │
│   │   │ • Config/feature     │     │ • Recent txns (top5) 30s     │  │   │
│   │   │   flags              │     │ • Risk scores        10m     │  │   │
│   │   │                      │     │ • JWT blacklist       15m    │  │   │
│   │   │ TTL: 10-30 min       │     │ • Distributed locks  10s     │  │   │
│   │   │ Max entries: 10K     │     │                              │  │   │
│   │   └──────────────────────┘     │ Eviction: LRU                │  │   │
│   │                                └──────────────────────────────┘  │   │
│   │                                                                  │   │
│   └──────────────────────────────────────────────────────────────────┘   │
│                                                                           │
│   LAYER 4: Database Query Cache                                           │
│   ┌──────────────────────────────────────────────────────────────────┐   │
│   │  • PostgreSQL shared_buffers — hot tables stay in memory         │   │
│   │  • Materialized views for reporting aggregates                   │   │
│   │  • Connection pooling via PgBouncer                              │   │
│   └──────────────────────────────────────────────────────────────────┘   │
│                                                                           │
└──────────────────────────────────────────────────────────────────────────┘
```

7.2 Cache Invalidation Strategy

Data Type	Pattern	Invalidation Trigger
Account Balance	**Cache-Aside**	Kafka `balance.changed` event → evict key
User Session	**Write-Through**	Login writes to Redis + DB simultaneously
Interest Rates	**TTL-Based**	Expires every 30 min; refresh from config DB
Transaction History	**Write-Behind**	New txn → update Redis list → async DB write
OTP Codes	**TTL auto-expire**	Redis `SET key value EX 300` — self-expiring
JWT Blacklist	**Write-Through**	Logout → add token hash to Redis blacklist

Critical rule: Financial data (balances, transactions) uses **short TTLs (30-60s)** with event-driven invalidation as a secondary guarantee. We never serve stale balance data for more than 60 seconds.

```java
@Cacheable(value = "account-balance", key = "#accountId", unless = "#result == null")
public BigDecimal getBalance(String accountId) {
    return accountRepository.findBalanceByAccountId(accountId);
}

@KafkaListener(topics = "account-events", groupId = "account-cache-cg")
public void onBalanceChanged(AccountEvent event) {
    if (event.getType() == BALANCE_CHANGED) {
        cacheManager.getCache("account-balance").evict(event.getAccountId());
    }
}
```

---

8. Multi-Level Filter Implementation

The transaction history screen requires complex filtering across multiple dimensions. This is implemented as a **composable filter pipeline** on both frontend and backend.

8.1 Filter Dimensions

```
┌──────────────────────────────────────────────────────────────────────┐
│                      MULTI-LEVEL FILTER UI                           │
│                                                                      │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────┐ │
│  │ LEVEL 1      │  │ LEVEL 2      │  │ LEVEL 3      │  │ LEVEL 4  │ │
│  │ Account      │  │ Date Range   │  │ Category     │  │ Amount   │ │
│  │              │  │              │  │              │  │ Range    │ │
│  │ ▼ Savings    │  │ From: _____  │  │ ☐ Credit     │  │          │ │
│  │ ▼ Current    │  │ To:   _____  │  │ ☐ Debit      │  │ Min: ____ │ │
│  │ ▼ Loan       │  │              │  │ ☐ UPI        │  │ Max: ____ │ │
│  │              │  │ Quick:       │  │ ☐ NEFT       │  │          │ │
│  │              │  │ • Today      │  │ ☐ RTGS       │  │ Quick:   │ │
│  │              │  │ • 7 days     │  │ ☐ Bill Pay   │  │ • < 1K   │ │
│  │              │  │ • 30 days    │  │ ☐ EMI        │  │ • 1K-10K │ │
│  │              │  │ • Custom     │  │ ☐ ATM        │  │ • 10K-1L │ │
│  │              │  │              │  │              │  │ • > 1L   │ │
│  └──────────────┘  └──────────────┘  └──────────────┘  └──────────┘ │
│                                                                      │
│  ┌──────────────────────────────────────────────┐                    │
│  │ LEVEL 5: Status            LEVEL 6: Search   │                    │
│  │ ○ All  ○ Success           [ keyword search ] │                    │
│  │ ○ Pending  ○ Failed                          │                    │
│  └──────────────────────────────────────────────┘                    │
│                                                                      │
│  Active Filters: [Savings x] [Last 30 days x] [UPI x] [Clear All]    │
└──────────────────────────────────────────────────────────────────────┘
```

8.2 Frontend Implementation (React)

Filters are managed as a **composable state object** with URL synchronization for shareable/bookmarkable filter states.

```
interface TransactionFilters {
  accountIds:    string[];
  dateRange:     { from: string; to: string } | null;
  categories:    TransactionCategory[];
  amountRange:   { min: number; max: number } | null;
  status:        'ALL' | 'SUCCESS' | 'PENDING' | 'FAILED';
  searchQuery:   string;
  page:          number;
  pageSize:      number;
  sortBy:        string;
  sortDir:       'asc' | 'desc';
}

function useTransactionFilters() {
  const [filters, setFilters] = useReducer(filterReducer, defaultFilters);
  const debouncedFilters = useDebounce(filters, 300);

  const { data, isLoading } = useQuery(
    ['transactions', debouncedFilters],
    () => transactionApi.search(debouncedFilters),
    { keepPreviousData: true, staleTime: 30_000 }
  );

  useSyncFiltersToURL(filters, setFilters);

  return { filters, setFilters, data, isLoading };
}
```

8.3 Backend Implementation (Spring Boot — Specification Pattern)

The backend translates filter params into composable **JPA Specifications** that chain together dynamically.

```
API Request                    Specification Builder              SQL Output
___________                    ______________________             __________

GET /api/v1/transactions
?accountIds=ACC-1,ACC-2   ──▶  ┌──────────────────┐
&from=2026-01-01          ──▶  │ AccountSpec      │──▶  WHERE account_id IN ( ... )
&to=2026-03-31            ──▶  │ DateRangeSpec    │──▶  AND created_at ≥  ...
&categories=UPI,NEFT      ──▶  │                  │──▶  AND created_at ≤  ...
&minAmount=1000           ──▶  │ CategorySpec     │──▶  AND category IN ( ... )
&maxAmount=50000          ──▶  │ AmountRangeSpec  │──▶  AND amount ≥ 1000
&status=SUCCESS           ──▶  │                  │──▶  AND amount ≤ 50000
&q=grocery                ──▶  │ StatusSpec       │──▶  AND status = 'SUCCESS'
&page=0&size=20           ──▶  │ SearchSpec       │──▶  AND description ILIKE '%grocery%'
&sort=createdAt,desc      ──▶  │ Pageable         │──▶  LIMIT 20 OFFSET 0
                               │ Sort             │──▶  ORDER BY created_at DESC
                               └──────────────────┘
```

```java
public class TransactionSpecBuilder {

    public static Specification<Transaction> build(TransactionFilterDTO f) {
        Specification<Transaction> spec = Specification.where(null);

        if (isNotEmpty(f.getAccountIds()))
            spec = spec.and(accountIdIn(f.getAccountIds()));

        if (f.getFrom() ≠ null && f.getTo() ≠ null)
            spec = spec.and(dateBetween(f.getFrom(), f.getTo()));

        if (isNotEmpty(f.getCategories()))
            spec = spec.and(categoryIn(f.getCategories()));

        if (f.getMinAmount() ≠ null || f.getMaxAmount() ≠ null)
            spec = spec.and(amountBetween(f.getMinAmount(), f.getMaxAmount()));

        if (f.getStatus() ≠ null && f.getStatus() ≠ Status.ALL)
            spec = spec.and(statusEquals(f.getStatus()));

        if (hasText(f.getSearchQuery()))
            spec = spec.and(descriptionContains(f.getSearchQuery()));

        return spec;
    }
}

public interface TransactionRepository
    extends JpaRepository<Transaction, UUID>, JpaSpecificationExecutor<Transaction> {}

public Page<TransactionDTO> search(TransactionFilterDTO filters, Pageable pageable) {
    Specification<Transaction> spec = TransactionSpecBuilder.build(filters);
    return transactionRepository.findAll(spec, pageable).map(mapper::toDTO);
}
```

---

9. Concurrency Control & Double-Spend Prevention

9.1 The Problem

In a high-throughput banking system, two concurrent requests can debit the same account simultaneously, potentially overdrawing the balance. This must be prevented at multiple levels.

9.2 Defense-in-Depth Strategy

```
┌──────────────────────────────────────────────────────────────────────┐
│                    CONCURRENCY CONTROL LAYERS                        │
│                                                                      │
│  LAYER 1: Idempotency Key (API Level)                                │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │  Client sends X-Idempotency-Key header with every mutation   │   │
│  │  Redis: SET NX idempotency:{key} EX 86400                    │   │
│  │  If key exists → return cached response (no reprocessing)    │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                                                                      │
│  LAYER 2: Distributed Lock (Application Level)                       │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │  Before modifying an account, acquire:                       │   │
│  │  Redis: SET account-lock:{accountId} {requestId} NX EX 10    │   │
│  │  Only one request can hold the lock at a time                │   │
│  │  Lock released after saga completes (or auto-expires)        │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                                                                      │
│  LAYER 3: Optimistic Locking (ORM Level)                             │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │  @Version column on accounts table                           │   │
│  │  UPDATE accounts SET balance = ?, version = version + 1      │   │
│  │  WHERE id = ? AND version = ?                                │   │
│  │  If 0 rows affected → OptimisticLockException → retry        │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                                                                      │
│  LAYER 4: Pessimistic Locking (Database Level)                       │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │  SELECT balance FROM accounts WHERE id = ? FOR UPDATE        │   │
│  │  Row-level lock held until transaction commits               │   │
│  │  Used ONLY for high-contention accounts (e.g., merchant)     │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                                                                      │
│  LAYER 5: Balance Check Constraint (Database Level)                  │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │  ALTER TABLE accounts ADD CONSTRAINT chk_balance_positive    │   │
│  │  CHECK (balance ≥ 0);                                        │   │
│  │  Last line of defense — DB rejects negative balances         │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

9.3 Implementation: Account Debit with All Layers

```java
@Transactional
public DebitResult debit(String accountId, BigDecimal amount, String idempotencyKey) {

    // Layer 1: Idempotency check
    Optional<String> cached = redisTemplate.opsForValue().get("idempotency:" + idempoten
    if (cached.isPresent()) return objectMapper.readValue(cached.get(), DebitResult.clas

    // Layer 2: Distributed lock
    String lockKey = "account-lock:" + accountId;
    String lockValue = UUID.randomUUID().toString();
    Boolean acquired = redisTemplate.opsForValue()
        .setIfAbsent(lockKey, lockValue, Duration.ofSeconds(10));

    if (!acquired) throw new AccountBusyException("Concurrent operation on account " + a

    try {
        // Layer 4: Pessimistic lock at DB level
        Account account = accountRepository.findByIdForUpdate(accountId)
            .orElseThrow(() -> new AccountNotFoundException(accountId));

        // Business validation
        if (account.getBalance().compareTo(amount) < 0) {
            throw new InsufficientBalanceException(accountId, amount, account.getBalance
        }

        account.setBalance(account.getBalance().subtract(amount));
        accountRepository.save(account);  // Layer 3: @Version check happens here

        DebitResult result = new DebitResult(accountId, amount, account.getBalance());

        // Cache result for idempotency
        redisTemplate.opsForValue()
            .set("idempotency:" + idempotencyKey, objectMapper.writeValueAsString(result
                 Duration.ofHours(24));

        return result;

    } finally {
        // Release distributed lock (only if we still own it)
        redisTemplate.execute(RELEASE_LOCK_SCRIPT, List.of(lockKey), lockValue);
    }
}
// Layer 5: CHECK (balance ≥ 0) fires if all above layers somehow fail
```

9.4 When to Use Which Lock

Scenario	Lock Strategy	Why
Normal account transfer	Distributed + Optimistic	Low contention, retry-friendly
Merchant account (high throughput)	Distributed + Pessimistic	High contention, avoid retry storms
Batch EMI debits (scheduler)	Pessimistic only	Sequential, no concurrency needed
Balance inquiry	No lock (read from cache)	Read-only, eventual consistency OK

10. Service Discovery & Inter-Service Communication

10.1 Communication Patterns

```
┌──────────────────────────────────────────────────────────────────────┐
│                INTER-SERVICE COMMUNICATION                           │
│                                                                      │
│  SYNCHRONOUS (request-response, latency-sensitive)                   │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │                                                              │   │
│  │  Internal service-to-service: gRPC (Protobuf)                │   │
│  │  ┌──────────┐    gRPC    ┌──────────┐                       │   │
│  │  │   Txn    │──────────→│ Account  │  • Type-safe contracts │   │
│  │  │ Service  │←──────────│ Service  │  • Binary, fast        │   │
│  │  └──────────┘            └──────────┘  • HTTP/2 multiplexing  │   │
│  │                                                              │   │
│  │  External / Client-facing: REST (JSON)                       │   │
│  │  ┌──────────┐    REST    ┌──────────┐                       │   │
│  │  │  Client  │──────────→│   BFF    │  • Human-readable      │   │
│  │  │          │←──────────│          │  • OpenAPI documented  │   │
│  │  └──────────┘            └──────────┘                       │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                                                                      │
│  ASYNCHRONOUS (fire-and-forget, high throughput)                     │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │                                                              │   │
│  │  Event-driven: Apache Kafka                                  │   │
│  │  ┌──────────┐  publish  ┌───────┐  consume  ┌──────────┐    │   │
│  │  │   Txn    │──────────→│ Kafka │──────────→│ Notific. │    │   │
│  │  │ Service  │           │       │──────────→│ Fraud    │    │   │
│  │  └──────────┘           └───────┘──────────→│ Audit    │    │   │
│  │                                              └──────────┘    │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                                                                      │
│  REAL-TIME (push to client)                                          │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │  WebSocket (STOMP over SockJS)                               │   │
│  │  ┌──────────┐  Kafka   ┌──────────┐  WS push  ┌──────────┐  │   │
│  │  │ Service  │─────────→│   BFF    │──────────→│  Client  │  │   │
│  │  └──────────┘          └──────────┘           └──────────┘  │   │
│  └──────────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────┘
```

10.2 Service Discovery

- **Kubernetes DNS** is the primary mechanism: each service is accessible at `{service-name}.{namespace}.svc.cluster.local`.
- **Istio service mesh** provides: mTLS encryption, traffic routing, canary deployments, and automatic retries.

- No external service registry (Eureka/Consul) needed — Kubernetes handles it natively.

10.3 gRPC Contract Example

```
// account-service.proto
service AccountService {
  rpc GetBalance (BalanceRequest) returns (BalanceResponse);
  rpc Debit (DebitRequest) returns (DebitResponse);
  rpc Credit (CreditRequest) returns (CreditResponse);
}

message DebitRequest {
  string account_id = 1;
  string amount = 2;         // String to avoid floating-point issues
  string currency = 3;
  string idempotency_key = 4;
  string correlation_id = 5;
}
```

11. Resilience Patterns

11.1 Pattern Overview

```
┌──────────────────────────────────────────────────────────────────────┐
│                    RESILIENCE PATTERNS (Resilience4j)                 │
│                                                                       │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │ CIRCUIT BREAKER                                              │    │
│  │                                                              │    │
│  │  CLOSED ──(failure rate > 50%)──▶ OPEN ──(wait 30s)──▶ HALF-OPEN  │
│  │    ▲                                                  │  │  │    │
│  │    └──────────(success in half-open)──────────────────┘  │  │    │
│  │                                                              │    │
│  │  Config per downstream service:                              │    │
│  │  • Account Service:  failure-rate=50%, wait=30s, calls=10    │    │
│  │  • Payment Gateway:  failure-rate=40%, wait=60s, calls=5     │    │
│  │  • Credit Bureau:    failure-rate=30%, wait=120s, calls=3    │    │
│  └──────────────────────────────────────────────────────────────┘    │
│                                                                       │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │ BULKHEAD (Thread Pool Isolation)                             │    │
│  │                                                              │    │
│  │  Each downstream dependency gets its own thread pool:        │    │
│  │  • account-service-pool:  max=20 threads, queue=50           │    │
│  │  • payment-gateway-pool:  max=10 threads, queue=20           │    │
│  │  • credit-bureau-pool:    max=5 threads, queue=10            │    │
│  │                                                              │    │
│  │  If pool exhausted → BulkheadFullException → fallback        │    │
│  │  Prevents one slow dependency from consuming all threads     │    │
│  └──────────────────────────────────────────────────────────────┘    │
│                                                                       │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │ RETRY (per dependency)                                       │    │
│  │                                                              │    │
│  │  • Max attempts: 3                                           │    │
│  │  • Backoff: exponential (100ms → 200ms → 400ms)              │    │
│  │  • Retry on: TimeoutException, IOException, 503              │    │
│  │  • Do NOT retry on: 4xx errors, InsufficientBalance          │    │
│  └──────────────────────────────────────────────────────────────┘    │
│                                                                       │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │ TIMEOUT                                                      │    │
│  │                                                              │    │
│  │  • Internal gRPC calls:     2 seconds                        │    │
│  │  • External payment APIs:   10 seconds                       │    │
│  │  • Credit bureau API:       15 seconds                       │    │
│  │  • Database queries:        5 seconds                        │    │
│  └──────────────────────────────────────────────────────────────┘    │
│                                                                       │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │ FALLBACKS                                                    │    │
│  │                                                              │    │
```

```
│  │  • Balance check fails → show "Last known: ₹XX (cached)"  │  │
│  │  • Notification fails → queue for retry (Kafka)           │  │
│  │  • Fraud check fails  → allow txn but flag for manual review│ │
│  │  • Credit bureau down → reject loan (fail-safe)           │  │
│  └──────────────────────────────────────────────────────────┘  │
└────────────────────────────────────────────────────────────────┘
```

11.2 Configuration (application.yml)

```yaml
resilience4j:
  circuitbreaker:
    instances:
      accountService:
        failure-rate-threshold: 50
        wait-duration-in-open-state: 30s
        sliding-window-size: 10
        permitted-number-of-calls-in-half-open-state: 3
      paymentGateway:
        failure-rate-threshold: 40
        wait-duration-in-open-state: 60s
        sliding-window-size: 5

  bulkhead:
    instances:
      accountService:
        max-concurrent-calls: 20
        max-wait-duration: 500ms

  retry:
    instances:
      accountService:
        max-attempts: 3
        wait-duration: 100ms
        exponential-backoff-multiplier: 2
        retry-exceptions:
          - java.io.IOException
          - java.util.concurrent.TimeoutException

  timelimiter:
    instances:
      accountService:
        timeout-duration: 2s
      creditBureau:
        timeout-duration: 15s
```

12. Security Architecture

12.1 Security Overview Diagram

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        SECURITY ARCHITECTURE                             │
│                                                                          │
│  ┌──────────────────────────────────────────────────────────────────┐   │
│  │ PERIMETER SECURITY                                               │   │
│  │                                                                  │   │
│  │  Internet ──► AWS WAF ──► CloudFront (CDN) ──► ALB ──► K8s    │  │
│  │                  │                                               │   │
│  │                  ├── SQL injection rules                         │   │
│  │                  ├── XSS rules                                   │   │
│  │                  ├── Rate limiting (IP-based)                    │   │
│  │                  ├── Geo-blocking (non-operating countries)      │   │
│  │                  └── Bot detection                               │   │
│  └──────────────────────────────────────────────────────────────────┘   │
│                                                                          │
│  ┌──────────────────────────────────────────────────────────────────┐   │
│  │ NETWORK SECURITY                                                 │   │
│  │                                                                  │   │
│  │  ┌──────────┐     ┌──────────────┐     ┌──────────────────┐     │   │
│  │  │ Public   │     │ Private      │     │ Data Subnet      │     │   │
│  │  │ Subnet   │     │ Subnet       │     │ (Isolated)       │     │   │
│  │  │          │     │              │     │                  │     │   │
│  │  │ • ALB    │──►  │ • API Gw     │──►  │ • PostgreSQL     │     │   │
│  │  │ • NAT Gw │     │ • Services   │     │ • Redis          │     │   │
│  │  │          │     │ • Kafka      │     │ • MongoDB        │     │   │
│  │  └──────────┘     └──────────────┘     └──────────────────┘     │   │
│  │                                                                  │   │
│  │  • No direct internet access to services or databases            │   │
│  │  • Security groups: whitelist only required ports                │   │
│  │  • VPC peering for cross-account access                          │   │
│  └──────────────────────────────────────────────────────────────────┘   │
│                                                                          │
│  ┌──────────────────────────────────────────────────────────────────┐   │
│  │ ENCRYPTION                                                       │   │
│  │                                                                  │   │
│  │  In Transit:                                                     │   │
│  │  • TLS 1.3 for all external traffic (A+ SSL Labs rating)        │   │
│  │  • mTLS (Istio) for all internal service-to-service traffic     │   │
│  │  • TLS for Redis, PostgreSQL, Kafka connections                  │   │
│  │                                                                  │   │
│  │  At Rest:                                                        │   │
│  │  • AES-256 for database storage (AWS KMS managed keys)          │   │
│  │  • S3 server-side encryption for statements/backups              │   │
│  │  • EBS volume encryption for all EC2/EKS nodes                   │   │
│  │                                                                  │   │
│  │  Field-Level (Application Layer):                                │   │
│  │  • Aadhaar numbers: AES-256-GCM, key in Vault                   │   │
│  │  • PAN card: AES-256-GCM, key in Vault                          │   │
│  │  • Card numbers: tokenized (never stored in plaintext)           │   │
```

```
│   │  • Phone/email: encrypted, searchable via HMAC index          │  │
│   └──────────────────────────────────────────────────────────────┘  │
│                                                                      │
│   ┌──────────────────────────────────────────────────────────────┐  │
│   │ SECRET MANAGEMENT (HashiCorp Vault)                           │  │
│   │                                                               │  │
│   │  ┌──────────────────────────────────────────────────────────┐  │  │
│   │  │  Vault Server (HA, unsealed with Shamir's Secret Sharing)│  │  │
│   │  │                                                          │  │  │
│   │  │  Secrets stored:                                         │  │  │
│   │  │  • Database credentials (rotated every 24h)              │  │  │
│   │  │  • JWT signing keys (RS256, rotated every 90 days)       │  │  │
│   │  │  • Kafka TLS certificates                                │  │  │
│   │  │  • Third-party API keys (NPCI, CIBIL, Twilio)            │  │  │
│   │  │  • Field-level encryption keys                           │  │  │
│   │  │                                                          │  │  │
│   │  │  Access: Kubernetes auth backend (pod identity)          │  │  │
│   │  │  Audit: Every secret access logged                       │  │  │
│   │  └──────────────────────────────────────────────────────────┘  │  │
│   └──────────────────────────────────────────────────────────────┘  │
│                                                                      │
│   ┌──────────────────────────────────────────────────────────────┐  │
│   │ APPLICATION SECURITY                                          │  │
│   │                                                               │  │
│   │  API Gateway enforces:                                        │  │
│   │  • CORS: whitelist only app domains                          │  │
│   │  • CSRF: double-submit cookie pattern                        │  │
│   │  • Content-Security-Policy, X-Frame-Options headers          │  │
│   │  • Request body size limit: 1MB                              │  │
│   │  • Input validation: Jakarta Bean Validation on all DTOs     │  │
│   │  • SQL injection: parameterized queries only (JPA Criteria)  │  │
│   │                                                               │  │
│   │  Authentication & Authorization:                              │  │
│   │  • OAuth 2.0 + OpenID Connect                                │  │
│   │  • JWT (RS256, 15-min expiry) + Refresh Token (7-day, Redis) │  │
│   │  • RBAC: CUSTOMER, TELLER, MANAGER, ADMIN, AUDITOR           │  │
│   │  • Method-level: @PreAuthorize("hasRole('ADMIN')")           │  │
│   │  • Account lockout: 5 failed attempts → 15 min lock          │  │
│   │                                                               │  │
│   │  Data Masking (in logs and API responses):                    │  │
│   │  • Account number: XXXX-XXXX-1234 (show last 4 only)        │  │
│   │  • Aadhaar: XXXX-XXXX-5678                                  │  │
│   │  • Phone: +91-XXXXX-67890                                   │  │
│   │  • Custom Logback pattern layout strips PII from all logs    │  │
│   └──────────────────────────────────────────────────────────────┘  │
│                                                                      │
│   ┌──────────────────────────────────────────────────────────────┐  │
│   │ PCI-DSS COMPLIANCE CONTROLS                                   │  │
│   │                                                               │  │
│   │  Requirement  │ Implementation                                │  │
│   │  ─────────────┼──────────────────────────────────────────     │  │
│   │  Req 1: FW    │ VPC security groups, WAF rules                │  │
│   │  Req 3: Data  │ AES-256 encryption, tokenization              │  │
│   │  Req 4: Transit│ TLS 1.3, mTLS                                │  │
│   │  Req 6: Secure│ SAST (SonarQube), DAST (OWASP ZAP)           │  │
│   │  Req 7: Access│ RBAC, least-privilege, maker-checker         │  │
```

```
│   │  Req 8: Auth   │ MFA, strong passwords, account lockout        │   │
│   │  Req 10: Log   │ Comprehensive audit trail in MongoDB          │   │
│   │  Req 11: Test │ Quarterly penetration testing                 │   │
│   │  Req 12: Policy│ Security policy docs, annual training        │   │
│   └──────────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────┘
```

---

13. API Design Contract

13.1 Versioning Strategy

- URL-based versioning: `/api/v1/...`, `/api/v2/...`
- **Deprecation policy:** old version supported for 6 months after new version launch.
- `Sunset` HTTP header warns clients of deprecation: `Sunset: Sat, 15 Oct 2026 00:00:00 GMT`
- Breaking changes (field removal, type change) → new major version.
- Non-breaking changes (new optional field) → same version.

13.2 Standard Error Response

All APIs return errors in a consistent envelope:

```json
{
  "status": 400,
  "error": "INSUFFICIENT_BALANCE",
  "message": "Account ACC-001 has insufficient balance for this transaction.",
  "details": [
    { "field": "amount", "issue": "Requested ₹10,000 but available balance is ₹3,500" }
  ],
  "correlationId": "req-789-abc",
  "traceId": "jaeger-trace-def",
  "timestamp": "2026-04-15T10:30:00Z",
  "path": "/api/v1/transfers",
  "documentation": "https://docs.bank.com/errors/INSUFFICIENT_BALANCE"
}
```

13.3 Standard Error Codes

HTTP Status	Error Code	When Used
400	VALIDATION_ERROR	Invalid input (missing field, wrong format)
400	INSUFFICIENT_BALANCE	Balance too low for transfer
401	AUTHENTICATION_FAILED	Invalid/expired token
403	ACCESS_DENIED	User lacks required role
404	RESOURCE_NOT_FOUND	Account/txn/loan not found
409	DUPLICATE_REQUEST	Idempotency key already used
409	ACCOUNT_LOCKED	Account frozen or locked
423	CONCURRENT_MODIFICATION	Optimistic lock failure
429	RATE_LIMIT_EXCEEDED	Too many requests
500	INTERNAL_ERROR	Unexpected server error
502	UPSTREAM_FAILURE	Dependency (payment gateway) failed
503	SERVICE_UNAVAILABLE	Circuit breaker open

13.4 Rate Limiting Tiers

Consumer Type	Limit	Window	Enforcement
Customer (web)	100 requests	1 min	Per user, sliding window
Customer (mobile)	60 requests	1 min	Per device token
Internal service	5,000 requests	1 min	Per service identity
Third-party API	1,000 requests	1 min	Per API key
Admin portal	200 requests	1 min	Per admin user

13.5 API Documentation

- **OpenAPI 3.0** spec auto-generated from Spring Boot annotations (`springdoc-openapi`).
- Swagger UI available at `/swagger-ui.html` (dev/staging only, disabled in prod).
- API changelog maintained per version in `/docs/api-changelog-v1.md`.

---

14. Frontend Architecture

14.1 React Application Structure

```
src/
├── app/
│   ├── store.ts                    # Redux Toolkit store
│   ├── routes.tsx                  # React Router definitions
│   └── ErrorBoundary.tsx           # Global error boundary
├── features/
│   ├── auth/
│   │   ├── components/             # LoginForm, MFADialog, BiometricPrompt
│   │   ├── hooks/                  # useAuth, useSession, useIdleTimeout
│   │   ├── services/               # authApi (RTK Query)
│   │   └── slice.ts
│   ├── accounts/
│   ├── transactions/
│   │   ├── components/
│   │   │   ├── TransactionList.tsx
│   │   │   ├── FilterPanel.tsx
│   │   │   ├── FilterChips.tsx
│   │   │   └── TransactionDetail.tsx
│   │   ├── hooks/
│   │   │   ├── useTransactionFilters.ts
│   │   │   └── useSyncFiltersToURL.ts
│   │   └── services/
│   │       └── transactionApi.ts
│   ├── payments/
│   ├── loans/
│   └── disputes/
├── shared/
│   ├── components/                 # Button, Modal, DataTable, Charts
│   ├── hooks/                      # useDebounce, usePagination, useWebSocket
│   └── utils/                      # formatCurrency, dateUtils, maskAccountNo
├── security/
│   ├── axiosInterceptor.ts         # Auto-attach JWT, refresh on 401
│   ├── idleTimeout.ts              # Auto-logout after 5 min inactivity
│   └── sensitiveDataGuard.ts       # Clear memory on logout, block localStorage
└── config/
    └── api.ts                      # Base URL, feature flags
```

14.2 Key Frontend Patterns

- **State Management:** Redux Toolkit + RTK Query for server-state caching with automatic revalidation.
- **Optimistic Updates:** Balance displays update immediately on transfer, roll back on failure.
- **WebSocket:** Real-time notifications via STOMP over SockJS, consuming from a user-specific Kafka topic.
- **Security:**

  - All sensitive data cleared from memory on logout; no financial data in `localStorage`.
  - Session idle timeout: auto-logout after 5 minutes of inactivity.
  - JWT auto-refresh: 401 interceptor silently refreshes token and retries the request.
  - Account numbers masked in UI by default; click-to-reveal with re-authentication.
- **Accessibility:** WCAG 2.1 AA compliance — keyboard navigation, screen reader support, high contrast mode.
- **Error Handling:** Global `ErrorBoundary` catches render errors; RTK Query `onError` handles API failures with user-friendly toasts.

---

15. Observability & Monitoring

15.1 Observability Stack

```
┌──────────────────────────────────────────────────────────────────────┐
│                        OBSERVABILITY STACK                           │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │ METRICS (Prometheus + Grafana)                               │   │
│  │                                                              │   │
│  │  Collected via Micrometer from every Spring Boot service:    │   │
│  │  • JVM: heap, GC, threads                                    │   │
│  │  • HTTP: request rate, latency (p50/p95/p99), error rate     │   │
│  │  • Kafka: consumer lag, produce rate, DLQ count              │   │
│  │  • Redis: hit rate, miss rate, connection pool               │   │
│  │  • DB: connection pool, query latency, active transactions   │   │
│  │  • Business: txns/sec, transfer volume, failed txns          │   │
│  │  • Circuit breaker: state transitions, rejection count       │   │
│  │                                                              │   │
│  │  Dashboards:                                                 │   │
│  │  • Service Health (per service)                              │   │
│  │  • Kafka Operations                                          │   │
│  │  • Business KPI (real-time txn volume, success rate)        │   │
│  │  • Infrastructure (CPU, memory, disk, network)               │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │ LOGGING (ELK Stack — Elasticsearch + Logstash + Kibana)      │   │
│  │                                                              │   │
│  │  • Structured JSON logs (Logback + LogstashEncoder)          │   │
│  │  • Every log line includes: correlationId, traceId, userId   │   │
│  │  • PII masking: custom Logback layout strips sensitive fields │   │
│  │  • Log levels: ERROR → PagerDuty, WARN → Slack, INFO → ELK   │   │
│  │  • Retention: 30 days hot, 90 days warm, 1 year cold (S3)    │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │ DISTRIBUTED TRACING (Jaeger)                                 │   │
│  │                                                              │   │
│  │  Propagation: W3C TraceContext headers                       │   │
│  │                                                              │   │
│  │  Client ──→ Gateway ──→ BFF ──→ Txn Service ──→ Account Svc  │   │
│  │    │           │          │          │                │      │   │
│  │    └───────────┴──────────┴──────────┴────────────────┘      │   │
│  │          All spans share the same traceId                    │   │
│  │                                                              │   │
│  │  Kafka: traceId injected into message headers,               │   │
│  │         consumer creates child span linking to producer span │   │
│  │                                                              │   │
│  │  Sampling: 10% in prod, 100% in staging                      │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐   │
```

```
│   │ ALERTING                                                    │   │
│   │                                                              │   │
│   │  Tool: Prometheus Alertmanager → PagerDuty / Slack / Email    │   │
│   │                                                              │   │
│   │  Critical (PagerDuty, page on-call):                         │   │
│   │  • Error rate > 5% for 2 minutes                             │   │
│   │  • p99 latency > 2s for 5 minutes                            │   │
│   │  • Kafka consumer lag > 10,000 for 5 minutes                 │   │
│   │  • DLQ message count > 0                                     │   │
│   │  • Circuit breaker OPEN for any service                      │   │
│   │  • Database replication lag > 10s                            │   │
│   │                                                              │   │
│   │  Warning (Slack #ops-alerts):                                │   │
│   │  • CPU > 80% for 10 minutes                                  │   │
│   │  • Memory > 85%                                              │   │
│   │  • Disk > 75%                                                │   │
│   │  • Certificate expiry < 30 days                              │   │
│   └──────────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────┘
```

15.2 Health Check Endpoints

Every service exposes:

Endpoint	Purpose
`/actuator/health`	Liveness probe (K8s) — is the JVM alive?
`/actuator/health/readiness`	Readiness probe — can it serve traffic?
`/actuator/health/db`	Database connectivity check
`/actuator/health/redis`	Redis connectivity check
`/actuator/health/kafka`	Kafka broker connectivity check
`/actuator/prometheus`	Metrics endpoint scraped by Prometheus

Readiness probe fails if DB or Redis is unreachable → Kubernetes removes pod from service endpoint → no traffic routed to unhealthy pod.

15.3 SLIs and SLOs

Service	SLI (Indicator)	SLO (Objective)
Transfer API	p99 latency	< 500ms
Transfer API	Success rate	> 99.9%
Balance API	p99 latency	< 200ms
Login API	p99 latency	< 1s
Notification	Delivery within	< 30 seconds
Kafka consumer	Processing lag	< 5,000 messages
Overall system	Availability	99.95% (monthly)

---

16. CI/CD Pipeline & Testing Strategy

16.1 Pipeline Architecture

```
┌─────────┐    ┌─────────┐    ┌─────────┐    ┌─────────┐    ┌─────────┐
│  Code   │    │  Build  │    │  Test   │    │ Staging │    │   Prod  │
│  Push   │──→│  & Scan │──→│  Suite  │──→│  Deploy │──→│  Deploy │
└─────────┘    └─────────┘    └─────────┘    └─────────┘    └─────────┘
     │              │              │              │              │
 GitHub PR     ┌──────┴──────┐  ┌──────┴──────┐  ┌──────┴──────┐  ┌──────┴──────┐
               │• Compile    │  │• Unit       │  │• Deploy     │  │• Canary     │
               │• Docker     │  │• Integr.    │  │  to stg     │  │  10%        │
               │  build      │  │• Contract   │  │• Smoke      │  │• Monitor    │
               │• SAST       │  │• E2E        │  │  tests      │  │  30 min     │
               │  (Sonar)    │  │• Security   │  │• Perf       │  │• Full       │
               │• DAST       │  │  (ZAP)      │  │  tests      │  │  rollout    │
               │• License    │  │             │  │• Manual     │  │             │
               │  check      │  │             │  │  QA gate    │  │             │
               └─────────────┘  └─────────────┘  └─────────────┘  └─────────────┘

Tool: GitHub Actions (CI) + ArgoCD (CD — GitOps)
```

16.2 Testing Pyramid

```
                    ┌──────────┐
                    │   E2E    │        5% of tests
                    │(Cypress) │        Run: pre-deploy to staging
                    ├──────────┤
                 ┌──│Contract  │──┐     10% of tests
                 │  │ (Pact)   │  │     Run: on every PR
                 │  ├──────────┤  │
              ┌──│  │Integration│  │──┐  25% of tests
              │  │  │(Testcont.)│  │  │  Run: on every PR
              │  │  ├──────────┤  │  │
           ┌──│  │  │   Unit   │  │  │──┐ 60% of tests
           │  │  │  │(JUnit 5) │  │  │  │ Run: on every commit
           │  │  │  └──────────┘  │  │  │
           └──┴──┴────────────────┴──┴──┘
```

Level	Tool	What It Tests
Unit	JUnit 5 + Mockito	Business logic, specifications, mappers
Integration	Testcontainers	Service + real DB + real Redis + real Kafka
Contract	Pact	API contracts between consumer/producer services
E2E	Cypress	Full user flows: login → transfer → verify balance
Security	OWASP ZAP	Automated pen testing on staging
Performance	Gatling	Load test: 10K concurrent users on staging

16.3 Deployment Strategy

Environment	Strategy	Details
Dev	Direct deploy	Every PR merge auto-deploys to dev
Staging	Blue-Green	Full environment swap, smoke tests gate promotion
Prod	Canary	10% traffic → monitor 30 min → 50% → 100%
Rollback	Automatic	If error rate > 2% during canary → auto-rollback

ArgoCD watches the Git repo (GitOps). Merged Kubernetes manifests in `deploy/` trigger automatic sync to the target environment.

16.4 Database Migration in CI/CD

- Flyway runs on application startup (before readiness probe passes).

- **Staging validates migrations** before they reach prod.
- Backward-compatible only: new columns nullable, old columns removed in the *next* release after code stops using them.

---

17. Deployment & Infrastructure

17.1 Kubernetes Cluster Layout

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        AWS EKS CLUSTER (Multi-AZ)                        │
│                                                                          │
│  ┌──────────────────────────────────────────────────────────────────┐   │
│  │ Namespace: banking-prod                                          │   │
│  │                                                                  │   │
│  │  Ingress (NGINX) ──► API Gateway (3 pods, HPA 2-10)              │   │
│  │                      │                                           │   │
│  │      ┌───────────────┼───────────────┬───────────────┐           │   │
│  │      ▼               ▼               ▼               ▼           │   │
│  │  Auth (3 pods)   Account (3)     Txn (5 pods)    Payment (3)     │   │
│  │  Loan (2 pods)   Notify (3)     Fraud (2 pods)   Admin (2)       │   │
│  │  Report (2)      Scheduler (2)  Config (2)       BFF-Web (3)     │   │
│  │  BFF-Mobile (3)                                                  │   │
│  │                                                                  │   │
│  │  Kafka Cluster (3 brokers, r=3) ──► Schema Registry (2)          │   │
│  │  Redis Cluster (3 master + 3 replica, multi-AZ)                  │   │
│  │  PostgreSQL (RDS: Primary + 2 Read Replicas, multi-AZ)           │   │
│  │  MongoDB Atlas (Replica Set, 3 nodes)                            │   │
│  │  Elasticsearch (3 nodes)                                         │   │
│  │  Vault (HA, 3 nodes)                                             │   │
│  └──────────────────────────────────────────────────────────────────┘   │
│                                                                          │
│  ┌──────────────────────────────────────────────────────────────────┐   │
│  │ Namespace: banking-monitoring                                    │   │
│  │                                                                  │   │
│  │  Prometheus (2) + Grafana (2) + Alertmanager (2)                 │   │
│  │  Jaeger (3)                                                      │   │
│  │  ELK Stack: Elasticsearch (3) + Logstash (2) + Kibana (2)        │   │
│  └──────────────────────────────────────────────────────────────────┘   │
│                                                                          │
│  ┌──────────────────────────────────────────────────────────────────┐   │
│  │ Namespace: banking-cicd                                          │   │
│  │                                                                  │   │
│  │  ArgoCD (2 pods)                                                 │   │
│  │  SonarQube (1 pod)                                               │   │
│  └──────────────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────────┘
```

17.2 Key NFRs (Non-Functional Requirements)

Metric	Target	How Achieved
Availability	99.95%	Multi-AZ, pod anti-affinity, circuit breakers
Latency (p99)	< 500ms	Redis caching, DB indexing, async processing
Throughput	10,000 txns/sec	Kafka partitioning, HPA auto-scaling
Recovery (RTO)	< 15 min	K8s self-healing, PG streaming replica, runbook
Recovery (RPO)	< 1 min	Synchronous PG replication, Kafka r=3
Data Durability	Zero loss	Kafka replication=3, PG synchronous commit
Security	PCI-DSS Level 1	See Section 12
Scalability	10x current load	HPA + Kafka partition scaling + read replicas

18. Disaster Recovery & Business Continuity

18.1 DR Architecture

```
┌──────────────────────────────┐          ┌──────────────────────────────┐
│       PRIMARY REGION         │          │       DR REGION              │
│       (ap-south-1 Mumbai)    │          │       (ap-south-2 Hyderabad) │
│                              │          │                              │
│  ┌────────────────────────┐  │          │  ┌────────────────────────┐  │
│  │ EKS Cluster (Active)   │  │          │  │ EKS Cluster (Standby)  │  │
│  │ • All services running │  │          │  │ • Scaled to 0 (warm)   │  │
│  │ • Handles 100% traffic │  │          │  │ • ArgoCD synced        │  │
│  └────────────────────────┘  │          │  └────────────────────────┘  │
│                              │          │                              │
│  ┌────────────────────────┐  │  async   │  ┌────────────────────────┐  │
│  │ PostgreSQL Primary     │──┼──repl.──►│  │ PostgreSQL Standby     │  │
│  │ (synchronous repl to   │  │          │  │ (async streaming repl) │  │
│  │  in-region replica)    │  │          │  │                        │  │
│  └────────────────────────┘  │          │  └────────────────────────┘  │
│                              │          │                              │
│  ┌────────────────────────┐  │  mirror  │  ┌────────────────────────┐  │
│  │ Kafka Cluster          │──┼──────────►│  │ Kafka Cluster (Mirror) │  │
│  │ (3 brokers)            │  │ Maker2   │  │ (3 brokers)            │  │
│  └────────────────────────┘  │          │  └────────────────────────┘  │
│                              │          │                              │
│  ┌────────────────────────┐  │  repl.   │  ┌────────────────────────┐  │
│  │ Redis Cluster          │──┼──────────►│  │ Redis Cluster          │  │
│  └────────────────────────┘  │          │  └────────────────────────┘  │
│                              │          │                              │
│  ┌────────────────────────┐  │          │  ┌────────────────────────┐  │
│  │ S3 (backups, stmts)    │──┼── CRR ──►│  │ S3 (cross-region repl) │  │
│  └────────────────────────┘  │          │  └────────────────────────┘  │
└──────────────────────────────┘          └──────────────────────────────┘

          Route 53 (DNS Failover)
          ┌──────────────────────┐
          │ Active-Passive       │
          │ Health check: /health│
          │ Failover TTL: 60s    │
          └──────────────────────┘
```

18.2 Recovery Objectives

Metric	Target	Mechanism
RTO (Recovery Time)	< 15 min	Pre-deployed DR cluster, DNS failover
RPO (Recovery Point)	< 1 min	Synchronous repl (in-region), async (cross)
RPO (cross-region)	< 5 min	Async PG replication + Kafka MirrorMaker2

18.3 Backup Strategy

Data Store	Backup Type	Frequency	Retention	Storage
PostgreSQL	Automated snapshot	Daily	30 days	S3 (encrypted)
PostgreSQL	WAL archiving	Continuous	7 days	S3
PostgreSQL	Logical dump	Weekly	90 days	S3 Glacier
MongoDB	Atlas auto-backup	Continuous	30 days	Atlas
Redis	RDB snapshot	Every 6 hours	7 days	S3
Kafka	Log retention	N/A	30-90 days	Broker disk
S3 (statements)	Cross-region repl	Continuous	7 years	S3 Glacier

18.4 DR Drill Schedule

- **Quarterly:** Tabletop exercise (walk-through, no actual failover)
- **Semi-annual:** Full failover drill to DR region, measure actual RTO/RPO
- **Annual:** Chaos engineering (Chaos Monkey — kill random pods, simulate AZ failure)

---

19. Regulatory Compliance

19.1 RBI Guidelines (India-Specific)

Regulation	Implementation
Data Localization	All data stored within India (ap-south-1/ap-south-2 only)
Transaction Limits	UPI: ₹1L/txn, NEFT: no limit, RTGS: ≥₹2L — enforced in Payment Service
Mandatory Reporting	CTR (Cash Transaction Report) for txns > ₹10L — auto-generated
Suspicious Activity Reporting	SAR auto-filed when Fraud Service scores > 0.8
KYC Requirements	Aadhaar eKYC + video KYC for account opening
Cooling Period	New beneficiary: 24h wait for transfers > ₹2L
Two-Factor Auth	Mandatory for all financial transactions (OTP/biometric)

19.2 Data Privacy (GDPR-aligned / India DPDP Act 2023)

Right	Implementation
Right to Access	GET `/api/v1/me/data-export` → generates full data package (JSON)
Right to Erasure	POST `/api/v1/me/delete` → anonymize PII, retain txn data for regulatory period
Consent Management	Explicit opt-in for marketing. Consent stored in `user_consents` table
Data Minimization	Only collect necessary data. Regular audit of stored fields
Breach Notification	Automated alert pipeline: detect → assess → notify regulator within 72h
Data Portability	Export in machine-readable JSON format

19.3 Data Retention Policy

Data Type	Active Retention	Archive	Deletion
Transaction records	3 years (hot)	7 years (S3 Glacier)	Purge after 10 years
Audit logs	1 year (MongoDB)	5 years (S3)	Purge after 7 years
User PII	Active account	90 days post-close	Anonymized after 90 days
Login logs	90 days (hot)	1 year (cold)	Purge after 1 year
KYC documents	Active account	8 years post-close	Per RBI mandate
Kafka events	30-90 days	—	Auto-deleted by retention policy
Statement PDFs	7 years	S3 Glacier	Purge after 10 years

19.4 Audit Trail

Every state-changing operation produces an audit event:

```json
{
  "auditId": "aud-123",
  "timestamp": "2026-04-15T10:30:00Z",
  "actor": {
    "userId": "user-007",
    "role": "CUSTOMER",
    "ip": "203.0.113.45",
    "userAgent": "Mozilla/5.0 ... ",
    "deviceId": "device-abc"
  },
  "action": "TRANSFER_INITIATED",
  "resource": {
    "type": "TRANSACTION",
    "id": "txn-456"
  },
  "details": {
    "fromAccount": "ACC-001 (masked)",
    "toAccount": "ACC-002 (masked)",
    "amount": 5000.00,
    "channel": "WEB"
  },
  "result": "SUCCESS",
  "correlationId": "req-789"
}
```

Stored in MongoDB (immutable collection, no update/delete operations allowed). Indexed by `userId`, `action`, `timestamp` for compliance queries.

---

20. End-to-End Request Lifecycle

```
User clicks "Transfer ₹5,000"
        │
        ▼
[1]  React dispatches action ──► RTK Query POST /api/v1/transfers
     (with X-Idempotency-Key header)
        │
        ▼
[2]  CDN passes through (no caching for mutations)
     WAF validates request (SQL injection, XSS rules)
        │
        ▼
[3]  API Gateway:
     ├── JWT validation (signature + expiry + blacklist check via Redis)
     ├── Rate limit check (Redis INCR, 100 req/min per user)
     ├── CORS/CSRF validation
     ├── Inject correlationId + traceId (Jaeger)
     └── Route to BFF-Web
        │
        ▼
[4]  BFF-Web:
     └── Forwards to Transaction Service (gRPC, with circuit breaker)
        │
        ▼
[5]  Transaction Service:
     ├── Idempotency check (Redis SET NX → first time? proceed)
     ├── Acquire distributed lock (Redis SET NX on accountId)
     ├── Start Saga:
     │   ├── Step 1: Debit source (gRPC → Account Service)
     │   │   └── Account Service: SELECT FOR UPDATE → check balance → debit → commit
     │   ├── Step 2: Credit destination (gRPC → Account Service)
     │   │   └── Account Service: SELECT FOR UPDATE → credit → commit
     │   └── Step 3: Persist saga state = COMPLETED
     ├── Release distributed lock
     └── PUBLISH event to Kafka "txn-events"
        │
        ├──► [6a] Notification Service CONSUMES ──► Sends SMS + Email + Push
        ├──► [6b] Fraud Service CONSUMES ──► Scores risk, flags if suspicious
        ├──► [6c] Audit Writer CONSUMES ──► Writes immutable audit log to MongoDB
        ├──► [6d] Cache Invalidation Consumer ──► Evicts balance cache for both accounts
        └──► [6e] Reporting Service CONSUMES ──► Updates materialized views
        │
        ▼
[7]  BFF-Web receives Kafka event via internal consumer
     └── Pushes real-time update to React via WebSocket
        │
        ▼
[8]  React UI:
     ├── Shows "Transfer Successful" toast
     ├── Invalidates balance query (RTK Query refetch)
     └── Adds new transaction to list (optimistic insert)
```

```
    Total latency (p99): < 500ms for steps 1–5
    Notification delivery: < 30s for step 6a
```

End of Document — Version 2.0